AML Client Risk Rating Model

Group members: Jaime Corzo Galdó, Wenyi (Betty) Li, Zihan Zhao

We built a binary client risk rating (CRR) model that classifies each customer as high risk or non-high risk. The approach is a random forest to rank and select features, followed by a ridge (L2-penalised) logistic regression as the final model. The final model uses 19 features, and every coefficient points in the direction AML logic would expect. Its cross-validated AUC on the training set is 0.762. On the hold-out test set, which we used only once at the very end, it has an AUC of 0.824 (95% bootstrap interval 0.75 to 0.89) and catches 83% of the true high-risk customers. All code is in the Appendix (Python, scikit-learn).

Data overview

Item	Value
Customers	588
High risk (rating = 1)	198 (33.7%)
Features	149 (the file actually has 150 columns, because tot_acct_num appears twice)
Missing values	none
Binary features	16
Continuous features	134, of which 114 have skewness above 5 (maximum 24)

All features were already standardised to mean 0 and standard deviation 1, including the binary flags.

1. Data transformation

Yes, a transformation is needed, but not another standardisation. Standardising only shifts and rescales a variable. It does not change the shape of its distribution. Most transaction features are zero for the majority of customers and very large for a few, so their distributions are extremely right-skewed. In a logistic regression a handful of extreme customers would then drive the coefficients.

We applied a signed log transform to the 134 continuous features and left the binary flags unchanged:

$$x' = \text{sign}\,(x)\ \ln\,(1 + |x|)$$

A plain log is not possible because the standardised values are negative for below-average customers. The signed version keeps the order and the sign but compresses the tails. The number of features with skewness above 5 fell from 114 to 94. The skew is not fully removed because many features are zero-inflated. Before fitting the logistic regression, the transformed features are standardised again so that the ridge penalty treats them equally.

We also removed columns that carry no extra information:

- 5 exact duplicates: tot_acct_num appears twice and is identical to prod_prim_acct_cnt, and the three domicile-country risk flags are identical to the three mailing-address flags.
- 10 near-constant columns, where more than 99% of customers share one value (mostly 3-month and 6-month wires to high and medium risk countries).

This leaves 135 features. These cleaning steps do not use the target, so it is safe to apply them before splitting the data.

2. Feature selection

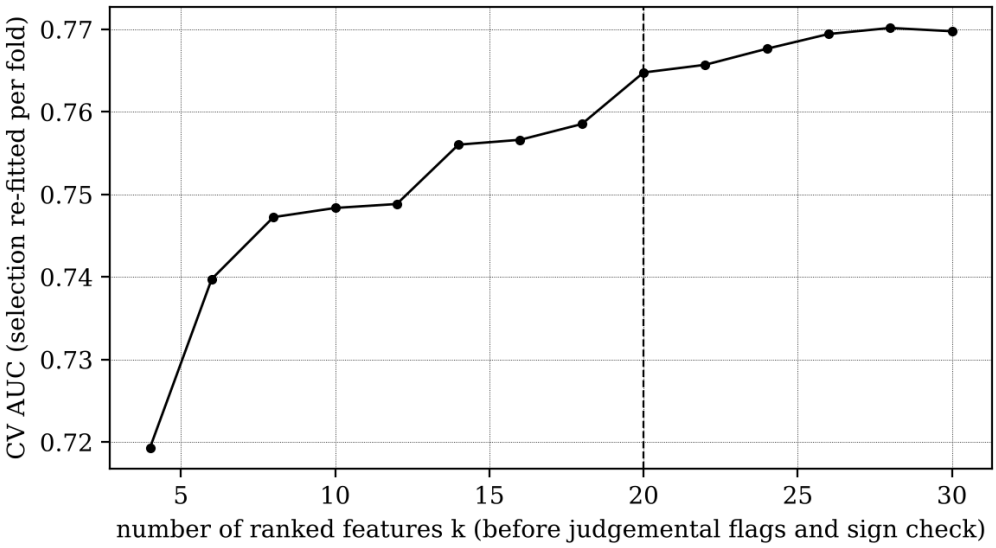
Yes, feature selection is necessary, for three reasons.

- Too many features for the sample. There are 135 features against 588 customers and only 138 high-risk customers in the training set.
- Strong multicollinearity. 285 feature pairs have $|r| > 0.9$, and 61 have $|r| > 0.99$. The same activity is measured over 3, 6 and 12 months and as a value, a count and an average. Collinear inputs make logistic regression coefficients unstable, and their signs can flip.
- Interpretability. A CRR model has to be explained to adjudicators and validators, so a shorter list of features with a clear AML meaning is preferable.

The selection pipeline has four steps:

1. Random forest ranking. A random forest (500 trees, minimum leaf size 3, balanced class weights) ranks the features by mean decrease in impurity.
2. Correlation filter. Going down the ranking, a feature is kept only if its correlation with every feature already kept is at most 0.8. This keeps the more important member of each correlated group.
3. Top k plus judgemental flags. We take the top k features and always add three flags with a clear AML meaning: high-risk mailing country, medium-risk product (prod_med) and Canadian residency. Impurity importance tends to rank binary flags low, because a flag offers only one split, so we do not leave them to the ranking.
4. Sign check, described in Question 6.

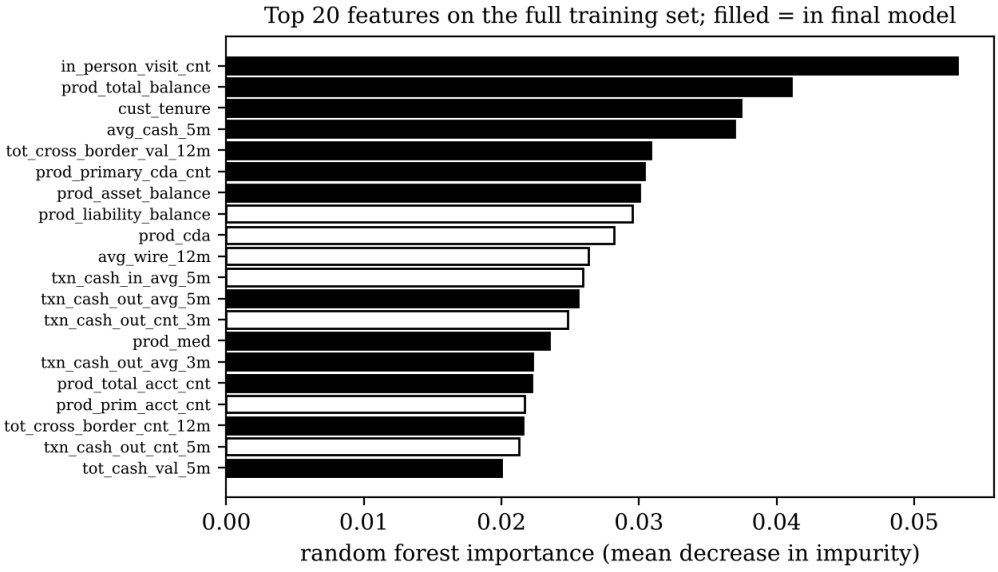
Choosing k without leakage. The ranking, the filter and the sign check all look at the target. If we ran them once on the whole training set and then cross-validated only the final ridge model, the validation folds would have helped choose the features, and the CV AUC would be too optimistic. So we re-ran the entire pipeline inside every fold. The random forest, the filter and the sign check are fitted on the fold's training part only, and the fold's validation part is used only for scoring. We used 5-fold stratified CV repeated 3 times (15 splits) and searched k from 4 to 30 together with C (Question 4).



Cross-validated AUC against k, with the selection re-fitted in every fold

The CV AUC rises quickly up to about k = 8, then slowly, reaching 0.770 at k = 28. We took the smallest k within 0.01 of the best, k = 20, because the remaining gain is small and more features make the model harder to explain.

The final feature list comes from running the same pipeline once on the full training set with k = 20. That gives 23 candidates (the top 20 plus the 3 flags). The sign check removes 4 of them, leaving 19.



Random forest importance on the full training set

3. Train / test split and sample weights

We used a stratified 70/30 split with a fixed random seed:

Set	Customers	High risk
Train	411	138 (33.6%)
Test	177	60 (33.9%)

Stratification keeps the share of high-risk customers the same in both sets. Every modelling decision was made with cross-validation on the training set: the number of features, C, the threshold, class weights and the comparison with other models. The test set was used once, to report the final model.

We did not set aside a separate validation set. With only 138 high-risk customers in training, a further split would leave fewer than 100 to fit the model. Repeated cross-validation does the job of a validation set while still letting every training customer be used for both fitting and validation.

We used sample weights through class_weight = "balanced". Each high-risk customer gets weight n / (2·n_1) ≈ 1.49 and each non-high customer gets n / (2·n_0) ≈ 0.75, so the two classes count equally. The main reason is the class imbalance (one high-risk customer for every two others): without weights the fitted probabilities lean toward the majority class, and at a 0.5 cut-off the model would miss most high-risk customers. Cross-validation confirms this. The CV AUC is almost the same with and without weights (0.762 against 0.760), because AUC only measures ranking. But at a 0.5 cut-off the out-of-fold recall is 63% with weights and only 35% without. The final cut-off is set separately in Question 4.

4. Model, hyperparameters and threshold

The final model is a ridge logistic regression. It estimates

$$P(\text{high} \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x)}}$$

by minimising the weighted log-loss plus an L2 penalty on the coefficients:

$$\min_{\beta} \; - \sum_i w_i \left[y_i \ln p_i + (1 - y_i) \ln (1 - p_i) \right] + \frac{1}{2C} \parallel \beta \parallel^2$$

The penalty shrinks coefficients toward zero. This reduces overfitting and stabilises the coefficients when features are correlated.

Hyperparameters, all tuned by the repeated CV above:

- k, the number of ranked features: 20 (Question 2).
- C, the inverse penalty strength, searched over 0.01, 0.03, 0.1, 0.3, 1 and 3 jointly with k. At k = 20 the CV AUC is between 0.762 and 0.765 for every C, so we took C = 0.01, the strongest penalty, which gives the most stable coefficients.
- The random forest settings used for ranking (500 trees, leaf size 3) were fixed, not tuned.

Threshold. The model outputs a probability, and we need a cut-off for "model high". We compared three cut-offs using out-of-fold predictions on the training set (averaged over the 3 CV repeats), not the test set:

Rule	Cut-off	OOF precision	OOF recall	OOF F1
Recall of at least 80%	0.420	0.474	0.800	0.595
Maximum F1	0.473	0.532	0.688	0.600
0.5	0.500	0.572	0.630	0.600

The three cut-offs give almost the same F1. We chose 0.420, the recall-based cut-off. A CRR model is a screening step: a customer flagged as high goes to enhanced due diligence, where an analyst can clear them. A missed high-risk customer is not reviewed at all. So we accept more false positives in order to catch 80% of the high-risk customers.

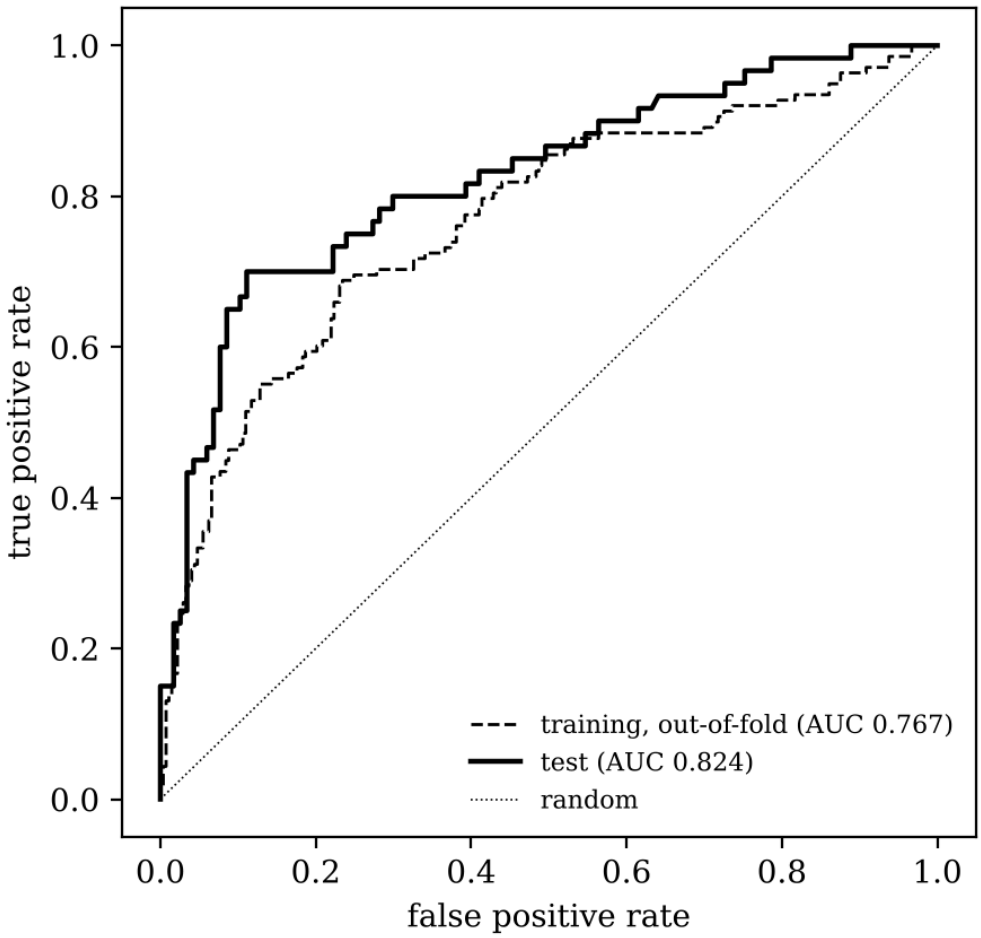
5. Performance

Cross-validation on the training set (15 splits, selection re-fitted in every fold) gives an AUC of 0.762, with a standard deviation of 0.042 across folds. The final model on the test set:

Measure	Test	95% CI (bootstrap)
ROC AUC	0.824	0.752 to 0.886
Gini (= 2·AUC − 1)	0.648	0.504 to 0.772
Precision (cut-off 0.420)	0.500	0.409 to 0.592
Recall	0.833	0.733 to 0.925
F1	0.625	0.538 to 0.703

The confidence intervals come from 2,000 bootstrap resamples of the test set: we draw 177 customers with replacement, recompute each metric, and take the 2.5% and 97.5% percentiles. Test confusion matrix:

	Model non-high	Model high
Actual non-high	67	50
Actual high	10	50



ROC curve: training out-of-fold and test

The test AUC (0.824) is above the CV AUC (0.762), but the test set has only 60 high-risk customers and its 95% interval runs from 0.75 to 0.89, which covers the CV value. The CV estimate averages over 15 splits, so it is the more reliable figure for new customers. Either way, the test result shows no sign of overfitting.

Comparison with other models. To see what the selection costs, we ran other models on the same 15 CV splits:

Model (CV on training set)	CV AUC	Difference vs final	Folds where better than final
Final model (selection + sign check)	0.762		
Same pipeline without the sign check	0.788	+0.027	87%
Ridge on all 135 features (C = 0.03)	0.814	+0.052	93%
Random forest on all 135 features	0.817	+0.055	87%

The final model gives up about 0.05 of AUC compared with using all features. About half of that comes from the sign check and the rest from keeping a shorter list. We explain why we accept this in Question 6.

6. Interpretation

Each coefficient is the change in log-odds for a one standard deviation increase in the (transformed) feature, and exp(β) is the corresponding odds ratio. The strong penalty (C = 0.01) keeps all coefficients small.

Feature	Coefficient	Odds ratio
tot_cross_border_cnt_12m (number of cross-border wires)	0.239	1.27
prod_primary_cda_cnt (chequing accounts)	0.163	1.18
prod_med (medium-risk product)	0.155	1.17
geo_mail_addr_country_rate_high (high-risk country)	0.152	1.16
txn_wires_in_avg_12m (average wire received)	0.144	1.16
avg_cash_5m (average cash transaction)	0.143	1.15
prod_primary_ted_cnt (term deposits)	−0.126	0.88
in_person_visit_cnt	0.117	1.12
geo_mail_addr_country_rate_low (low-risk country)	−0.085	0.92
cust_cdn_resident (Canadian resident)	−0.070	0.93
txn_cash_out_avg_5m	0.065	1.07
cust_tenure (years with the bank)	−0.047	0.95
tot_cross_border_val_12m	0.045	1.05
prod_primary_crc_cnt (credit cards)	0.040	1.04
prod_asset_balance	−0.038	0.96
prod_total_balance	0.031	1.03
tot_cash_val_5m	0.024	1.02
prod_total_acct_cnt	0.010	1.01
txn_cash_out_avg_3m	< 0.001	1.00

The strongest drivers are cross-border wire activity, holding chequing and medium-risk products, a mailing address in a high-risk country, and cash intensity. These are standard AML red flags. The features that lower the score also make sense: term deposits (a slow-moving savings product), a low-risk mailing country, Canadian residency and a longer relationship with the bank. The last two features in the table have been shrunk almost to zero and have no practical effect.

Counter-intuitive features. There are two cases.

- occu_h (high-risk occupation). In the raw data, 31.8% of customers with a high-risk occupation are rated high, against 35.4% of the others, so the flag points the wrong way. The random forest ranked it 41st, so it was never selected.
- Sign flips from collinearity. When many correlated cash variables enter together, some coefficients take the wrong sign. For example, more cash withdrawals would lower the risk score, because one cash coefficient offsets another. These are not real effects.

Possible reasons for the occupation result:

- Selection or survivorship bias. Customers in high-risk occupations already receive extra scrutiny at onboarding, so the riskiest ones may have been declined or exited, and the ones left in the data are cleaner than average.
- Label and adjudication effects. Adjudicators may have already adjusted for occupation, or the flag may be too coarse, covering many low-risk people in a broad category.
- Confounding with other features. occu_h is correlated with holding savings and chequing products ($|r| \approx 0.2$–0.26).

How we addressed it:

- Expected signs. Each feature gets an expected sign. This is its own correlation with the target, overridden by AML judgement where the meaning is clear: transaction activity, high-risk country and medium-risk product should raise risk; tenure and residency should lower it.
- Sign check. We repeatedly drop the least important feature whose ridge coefficient has the wrong sign and refit, until no wrong signs remain. In the final fit this removed four features: the number of cash withdrawals and cash deposits over 3 months, the value of cash withdrawals over 5 months, and the number of account types. Like the rest of the pipeline, the sign check is re-run inside every CV fold.
- Occupation. We left occu_h out of the model rather than let it lower the score. If the business wanted occupation in the model, the safer option would be an expert override, for example a minimum positive weight, rather than learning its sign from this data.

We also tried bounding each coefficient to its expected sign instead of dropping features, by fitting a sign-constrained ridge regression with L-BFGS-B. On the same CV splits it gives the same AUC as the sign check (0.762), so the choice between the two does not matter here.

Residency. Residency is a strong signal on its own: 77% of the 30 non-residents are rated high, against 31% of residents. It is in the final model with the expected negative sign. Its coefficient is small because it overlaps almost completely with the mailing-country flags. All 25 customers with a high-risk mailing country are non-residents, and residency has a correlation of -0.92 with the high-risk country flag. The ridge penalty spreads the shared effect across these correlated flags instead of giving it all to one of them.

Why we accept the cost of the selection. Using all features would add about 0.05 of CV AUC. We still prefer the 19-feature model, for three reasons.

- The extra accuracy comes partly from coefficients that contradict AML logic. Without the sign check, more cash withdrawals or more cash deposits would lower a customer's risk score. These relationships come from correlated variables offsetting each other. An adjudicator or validator could not defend them, and a customer could lower their own rating by moving more cash.
- Offsetting coefficients are fragile. They depend on the exact correlation between overlapping 3-month and 5-month cash variables, which can change as customer behaviour or data feeds change.
- A CRR model decides who gets enhanced due diligence, so its ratings must be explainable to regulators and in model validation. Conceptual soundness is a requirement, not a nice-to-have.

If more accuracy were needed, the better route would be to build features that capture the useful part of the offset directly, such as net cash flow or the ratio of cash out to cash in, rather than to keep contradictory coefficients.

7. Other issues and actions

Issue	Why it matters	Action
Feature selection uses the target	Choosing features on the whole training set and then cross-validating would leak the validation folds into the selection and overstate the CV AUC	Re-ran the full selection pipeline inside every CV fold
Test set reused for decisions	Comparing options on the test set would make the test result optimistic	All decisions made on CV; test set used once for the final model
Duplicate and identical columns	Perfect collinearity and double counting	Dropped 5 columns (Question 1)
Zero-inflated, near-constant features	Little information and unstable estimates	Dropped 10 columns with more than 99% one value
Small sample (138 high-risk customers in training)	High variance and overfitting	Ridge penalty, shorter feature list, repeated CV for every tuning choice; no separate validation set
Unstable selection at the margin	Features near the cut-off move in and out between folds	Checked how often each final feature is chosen across the 15 CV splits (see below)
Judgemental flags bypass the correlation filter	Residency, the low-risk country flag and the high-risk country flag are highly correlated (correlations up to 0.97)	Kept, because each has a clear meaning and the expected sign; ridge shrinks them together
Binary flags were standardised	Coefficients are per standard deviation, not per 0/1 change	Interpret through odds ratios per SD; the direction is unaffected
in_person_visit_cnt raises the score	May partly reflect the review process itself, since high-risk customers are often asked to come in for enhanced due diligence	Kept, but flagged as a possible feedback effect to check with the business before deployment
Small test set (60 high-risk customers)	Test metrics are noisy (AUC ±0.07)	Bootstrap 95% intervals; treat the CV AUC as the main estimate

Selection stability. The three judgemental flags are in every fold by construction. Of the 16 ranked features in the final model:

- 8 are chosen in at least 93% of the 15 CV splits: in-person visits, asset balance, total balance, tenure, average cash, cross-border wire value and count, and the 5-month average cash withdrawal.
- 5 are chosen in 60–80% of splits.
- Three are less stable: total cash value and the low-risk country flag (33% each), and the average incoming wire (never chosen inside the folds, only on the full training set).

These marginal features have small coefficients, so the ratings do not depend much on whether they are included.

Appendix: Python code

```
"""AML client risk rating model: random forest feature selection + ridge logistic regression.

All modelling choices (number of features k, penalty C, threshold, class weights, model variants)
are made with cross-validation on the training set. The whole selection pipeline (random forest
ranking, correlation filter, judgemental flags, sign check) is re-fitted inside every fold, so the
validation fold never influences which features are chosen. The test set is used once, at the end.
"""
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.model_selection import train_test_split, RepeatedStratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (roc_auc_score, roc_curve, precision_score, recall_score,
                             f1_score, confusion_matrix)

SEED = 42
plt.rcParams.update({"font.family": "serif", "font.size": 10})

# ============================================================== data and cleaning (no target use
d)
df = pd.read_excel("data.xlsx", sheet_name="data")
y = df["rating"].astype(int)
X = df.drop(columns=["cust_id_masked", "rating"])
print("customers:", len(df), " high risk:", y.sum(), f"({y.mean():.1%})", " columns:", X.shape[1])

# Q1: signed log transform of continuous features
binary = [c for c in X.columns if X[c].nunique() <= 2]
continuous = [c for c in X.columns if c not in binary]
Xt = X.copy()
Xt[continuous] = np.sign(X[continuous]) * np.log1p(np.abs(X[continuous]))
print("skew > 5 before / after:", int((X[continuous].skew() > 5).sum()), "/",
      int((Xt[continuous].skew() > 5).sum()))

# exact duplicates and near-constant columns
dup = Xt.T.duplicated()
print("duplicate columns dropped:", list(Xt.columns[dup]))
Xt = Xt.loc[:, ~dup]
mode_share = Xt.apply(lambda c: c.value_counts(normalize=True).iloc[0])
near_const = list(mode_share[mode_share > 0.99].index)
Xt = Xt.drop(columns=near_const)
print("near-constant dropped:", len(near_const), " features left:", Xt.shape[1])

# Q3: stratified 70/30 split; the test set is not touched until the final evaluation
X_tr, X_te, y_tr, y_te = train_test_split(Xt, y, test_size=0.3, stratify=y, random_state=SEED)
print(f"train {len(y_tr)} ({y_tr.sum()} high)   test {len(y_te)} ({y_te.sum()} high)")

# ============================================================== pipeline pieces
JUDGEMENT = ["geo_mail_addr_country_rate_high", "prod_med", "cust_cdn_resident"]
OVERRIDE = {"cust_cdn_resident": -1, "cust_tenure": -1, "geo_mail_addr_country_rate_high": 1, "prod
_med": 1}
ACTIVITY = ("txn_", "tot_cash", "tot_cross", "avg_")
```

```
CORR_CUT = 0.8

def ridge(C, cw="balanced"):
    return make_pipeline(StandardScaler(),
                         LogisticRegression(penalty="l2", C=C, class_weight=cw, max_iter=5000))

def rank_features(Xa, ya):
    """Random forest ranking followed by the correlation filter, fitted on (Xa, ya) only."""
    rf = RandomForestClassifier(n_estimators=500, min_samples_leaf=3, max_features="sqrt",
                                class_weight="balanced", random_state=SEED, n_jobs=-1).fit(Xa, ya)
    imp = pd.Series(rf.feature_importances_, index=Xa.columns).sort_values(ascending=False)
    corr = Xa.corr().abs()
    kept = []
    for f in imp.index:
        if all(corr.loc[f, k] <= CORR_CUT for k in kept):
            kept.append(f)
    return kept, imp

def expected_signs(Xa, ya, feats):
    s = {f: np.sign(np.corrcoef(Xa[f], ya)[0, 1]) for f in feats}
    s.update({f: 1 for f in feats if f.startswith(ACTIVITY)})
    s.update({f: v for f, v in OVERRIDE.items() if f in feats})
    return s

def select(Xa, ya, kept, imp, k, C, sign_check=True, cw="balanced"):
    """Top-k ranked features + judgemental flags, then drop wrong-sign features one at a time."""
    feats = kept[:k] + [f for f in JUDGEMENT if f not in kept[:k]]
    if not sign_check:
        return feats
    exp = expected_signs(Xa, ya, feats)
    while True:
        m = ridge(C, cw).fit(Xa[feats], ya)
        coef = pd.Series(m.named_steps["logisticregression"].coef_[0], index=feats)
        wrong = [f for f in feats if np.sign(coef[f]) != exp[f]]
        if not wrong:
            return feats
        feats.remove(min(wrong, key=lambda f: imp[f]))

class SignRidge:
    """Class-weighted L2 logistic regression with each coefficient bounded to its expected sign."""
    def __init__(self, C, signs):
        self.C, self.signs = C, signs
    def fit(self, Xa, ya):
        Xa, ya = np.asarray(Xa, float), np.asarray(ya, float)
        self.mu, self.sd = Xa.mean(0), Xa.std(0) + 1e-12
        Z = (Xa - self.mu) / self.sd
        w = np.where(ya == 1, len(ya) / (2 * ya.sum()), len(ya) / (2 * (len(ya) - ya.sum())))
        def obj(b):
            p = 1 / (1 + np.exp(-(b[0] + Z @ b[1:])))
            loss = -np.sum(w * (ya * np.log(p + 1e-12) + (1 - ya) * np.log(1 - p + 1e-12)))
            grad = np.r_[np.sum(w * (p - ya)), Z.T @ (w * (p - ya)) + b[1:] / self.C]
            return loss + 0.5 * np.sum(b[1:] ** 2) / self.C, grad
        bounds = [(None, None)] + [((0, None) if s > 0 else (None, 0)) for s in self.signs]
        self.b = minimize(obj, np.zeros(Z.shape[1] + 1), jac=True, method="L-BFGS-B", bounds=bound
s).x
        return self
    def predict_proba(self, Xa):
        z = self.b[0] + ((np.asarray(Xa, float) - self.mu) / self.sd) @ self.b[1:]
        p = 1 / (1 + np.exp(-z))
        return np.c_[1 - p, p]

# ============================================================ Q2 / Q4: tune k and C with fold-w
ise selection
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=SEED)
splits = list(cv.split(X_tr, y_tr))
print("CV splits:", len(splits), "(5 folds x 3 repeats)")
```

```
ks = list(range(4, 31, 2))
Cs = [0.01, 0.03, 0.1, 0.3, 1, 3]
fold_rank = []                                    # ranking re-fitted on each fold's training part
for tr_i, _ in splits:
    fold_rank.append(rank_features(X_tr.iloc[tr_i], y_tr.iloc[tr_i]))

def cv_eval(k, C, sign_check=True, cw="balanced", model="ridge", collect=False):
    """AUC on every CV split for the whole pipeline; optionally collect out-of-fold scores."""
    aucs, oof = [], np.zeros((cv.n_repeats, len(y_tr)))
    for s, ((tr_i, va_i), (kept, imp)) in enumerate(zip(splits, fold_rank)):
        Xa, ya, Xv, yv = X_tr.iloc[tr_i], y_tr.iloc[tr_i], X_tr.iloc[va_i], y_tr.iloc[va_i]
        if model == "signridge":
            feats = select(Xa, ya, kept, imp, k, C, sign_check=False)
            exp = expected_signs(Xa, ya, feats)
            m = SignRidge(C, [exp[f] for f in feats]).fit(Xa[feats], ya)
        else:
            feats = select(Xa, ya, kept, imp, k, C, sign_check, cw)
            m = ridge(C, cw).fit(Xa[feats], ya)
        p = m.predict_proba(Xv[feats])[:, 1]
        aucs.append(roc_auc_score(yv, p))
        oof[s // 5, va_i] = p
    return (np.array(aucs), oof) if collect else np.array(aucs)

grid = {(k, C): cv_eval(k, C).mean() for k in ks for C in Cs}
best = max(grid, key=grid.get)
# simplest model within 0.01 of the best: smallest k, then the strongest penalty (smallest C)
k_fin, C_fin = min([kc for kc, v in grid.items() if v >= grid[best] - 0.01], key=lambda kc: (kc[0],
kc[1]))
print(f"best (k, C) = {best}, CV AUC {grid[best]:.3f};  chosen (k, C) = ({k_fin}, {C_fin}), CV AUC
{grid[(k_fin, C_fin)]:.3f}")
auc_by_k = pd.Series({k: max(grid[(k, C)] for C in Cs) for k in ks})
print("CV AUC by k (best C for each k):", auc_by_k.round(3).to_dict())
print("CV AUC by C at chosen k:", {C: round(grid[(k_fin, C)], 3) for C in Cs})

# ================================================================ model variants compared on the sa
me CV splits
fin_aucs, fin_oof = cv_eval(k_fin, C_fin, collect=True)
variants = {
    "final pipeline (sign check)": fin_aucs,
    "no sign check": cv_eval(k_fin, C_fin, sign_check=False),
    "sign-constrained ridge instead of sign check": cv_eval(k_fin, C_fin, model="signridge"),
    "final pipeline without class weights": cv_eval(k_fin, C_fin, cw=None),
}
for name, a in variants.items():
    d = a - fin_aucs
    print(f"CV {name}: AUC {a.mean():.3f} (sd {a.std():.3f})  diff vs final {d.mean():+.3f}, better
in {np.mean(d > 0):.0%} of folds")

# benchmarks on the same splits (all 135 features, no selection)
bench = {"random forest, all features": [], "ridge, all features (C = 0.03)": []}
for tr_i, va_i in splits:
    Xa, ya, Xv, yv = X_tr.iloc[tr_i], y_tr.iloc[tr_i], X_tr.iloc[va_i], y_tr.iloc[va_i]
    rf = RandomForestClassifier(n_estimators=500, min_samples_leaf=3, class_weight="balanced",
                                random_state=SEED, n_jobs=-1).fit(Xa, ya)
    bench["random forest, all features"].append(roc_auc_score(yv, rf.predict_proba(Xv)[:, 1]))
    bench["ridge, all features (C = 0.03)"].append(roc_auc_score(yv, ridge(0.03).fit(Xa, ya).predic
t_proba(Xv)[:, 1]))
for name, a in bench.items():
    d = np.array(a) - fin_aucs
    print(f"CV benchmark {name}: AUC {np.mean(a):.3f}  diff vs final {d.mean():+.3f}, better in {n
p.mean(d > 0):.0%} of folds")

# ================================================================ threshold from out-of-fold scores
(training set only)
def cut_for_recall(yv, p, target=0.80):
```

```
    fpr, tpr, thr = roc_curve(yv, p)
    return thr[np.argmax(tpr >= target)]
thr_recall = np.mean([cut_for_recall(y_tr, fin_oof[r]) for r in range(cv.n_repeats)])
grid_t = np.linspace(0.05, 0.95, 181)
thr_f1 = np.mean([grid_t[np.argmax([f1_score(y_tr, (fin_oof[r] >= t).astype(int)) for t in grid_
t])]
                  for r in range(cv.n_repeats)])
def oof_metrics(oof, t):
    out = []
    for r in range(cv.n_repeats):
        pr = (oof[r] >= t).astype(int)
        out.append([precision_score(y_tr, pr), recall_score(y_tr, pr), f1_score(y_tr, pr)])
    return np.mean(out, axis=0)
for name, t in [("recall >= 80%", thr_recall), ("max F1", thr_f1), ("0.5", 0.5)]:
    pr, rc, f1 = oof_metrics(fin_oof, t)
    print(f"OOF threshold {name}: cut {t:.3f}  precision {pr:.3f} recall {rc:.3f} F1 {f1:.3f}")
nw_oof = cv_eval(k_fin, C_fin, cw=None, collect=True)[1]
print("OOF at 0.5 without class weights: precision/recall/F1", oof_metrics(nw_oof, 0.5).round(3),
      " with:", oof_metrics(fin_oof, 0.5).round(3))
threshold = thr_recall

# ================================================================ final fit on the whole training s
et
kept_all, imp_all = rank_features(X_tr, y_tr)
features = select(X_tr, y_tr, kept_all, imp_all, k_fin, C_fin)
model = ridge(C_fin).fit(X_tr[features], y_tr)
coef = pd.Series(model.named_steps["logisticregression"].coef_[0], index=features)
pre = kept_all[:k_fin] + [f for f in JUDGEMENT if f not in kept_all[:k_fin]]
print(f"final: {len(pre)} candidates -> {len(features)} after sign check; dropped: {[f for f in pre
if f not in features]}")
table = pd.DataFrame({"coefficient": coef.round(3), "odds_ratio_per_sd": np.exp(coef).round(2),
                      "rf_rank": [list(imp_all.index).index(f) + 1 for f in features]})
print(table.sort_values("coefficient", key=abs, ascending=False).to_string())

# selection stability: how often each final feature was selected inside the CV folds
counts = pd.Series(0, index=X_tr.columns)
for (tr_i, _), (kept, imp) in zip(splits, fold_rank):
    counts[select(X_tr.iloc[tr_i], y_tr.iloc[tr_i], kept, imp, k_fin, C_fin)] += 1
print("selection frequency of final features across folds:", (counts[features] / len(splits)).round
(2).to_dict())
print("other features selected in at least half the folds:",
      (counts.drop(features)[counts.drop(features) >= len(splits) / 2] / len(splits)).round(2).to_d
ict())

# ================================================================ Q6: occupation and residency
print(df.groupby("occu_h")["rating"].agg(["mean", "count"]).round(3).to_string())
print("occu_h rank in final random forest:", list(imp_all.index).index("occu_h") + 1, "of", len(imp
_all))
print(df.groupby("cust_cdn_resident")["rating"].agg(["mean", "count"]).round(3).to_string())
print(pd.crosstab(df["cust_cdn_resident"] < 0, df["geo_mail_addr_country_rate_high"] > 0,
                  rownames=["non-resident"], colnames=["high-risk mailing country"]))
print("corr(residency, country flag):", round(X_tr["cust_cdn_resident"].corr(X_tr["geo_mail_addr_cou
ntry_rate_high"]), 3))

# ================================================================ Q5: final evaluation on the test
set (used once)
p_te = model.predict_proba(X_te[features])[:, 1]
pred = (p_te >= threshold).astype(int)
tn, fp, fn, tp = confusion_matrix(y_te, pred).ravel()
auc_te = roc_auc_score(y_te, p_te)
print(f"TEST: AUC {auc_te:.3f} Gini {2*auc_te-1:.3f} precision {precision_score(y_te, pred):.3f} "
      f"recall {recall_score(y_te, pred):.3f} F1 {f1_score(y_te, pred):.3f}  TP={tp} FP={fp} FN={f
n} TN={tn}")
rng = np.random.default_rng(SEED)
yt = y_te.values
boot = {k: [] for k in ["AUC", "Gini", "precision", "recall", "F1"]}
```

```
for _ in range(2000):
    i = rng.integers(0, len(yt), len(yt))
    pr = (p_te[i] >= threshold).astype(int)
    a = roc_auc_score(yt[i], p_te[i])
    boot["AUC"].append(a); boot["Gini"].append(2 * a - 1)
    boot["precision"].append(precision_score(yt[i], pr)); boot["recall"].append(recall_score(yt[i],
pr))
    boot["F1"].append(f1_score(yt[i], pr))
for k, v in boot.items():
    print(f"bootstrap {k}: 95% CI [{np.percentile(v, 2.5):.3f}, {np.percentile(v, 97.5):.3f}]")

# ================================================================== figures (black and white)
fig, ax = plt.subplots(figsize=(6.5, 3.8))
top = imp_all.head(20)[::-1]
ax.barh(range(len(top)), top.values, color=["black" if f in features else "white" for f in top.inde
x],
        edgecolor="black")
ax.set_yticks(range(len(top))); ax.set_yticklabels(top.index, fontsize=7)
ax.set_xlabel("random forest importance (mean decrease in impurity)")
ax.set_title("Top 20 features on the full training set; filled = in final model", fontsize=10)
fig.tight_layout(); fig.savefig("fig_rf_importance.png", dpi=200); plt.close(fig)

fig, ax = plt.subplots(figsize=(6, 3.4))
ax.plot(auc_by_k.index, auc_by_k.values, color="black", marker="o", ms=3, lw=1)
ax.axvline(k_fin, color="black", ls="--", lw=0.8)
ax.set_xlabel("number of ranked features k (before judgemental flags and sign check)")
ax.set_ylabel("CV AUC (selection re-fitted per fold)")
ax.grid(color="black", lw=0.3, ls=":")
fig.tight_layout(); fig.savefig("fig_cv_k.png", dpi=200); plt.close(fig)

fpr_o, tpr_o, _ = roc_curve(y_tr, fin_oof[0])
fpr_t, tpr_t, _ = roc_curve(y_te, p_te)
fig, ax = plt.subplots(figsize=(4.6, 4.4))
ax.plot(fpr_o, tpr_o, color="black", lw=1, ls="--",
        label=f"training, out-of-fold (AUC {roc_auc_score(y_tr, fin_oof[0]):.3f})")
ax.plot(fpr_t, tpr_t, color="black", lw=1.6, label=f"test (AUC {auc_te:.3f})")
ax.plot([0, 1], [0, 1], color="black", lw=0.6, ls=":", label="random")
ax.set_xlabel("false positive rate"); ax.set_ylabel("true positive rate")
ax.legend(frameon=False, loc="lower right", fontsize=8)
fig.tight_layout(); fig.savefig("fig_roc.png", dpi=200); plt.close(fig)
```